pySimNMR: User Manual and Methods

A. P. Dioguardi and collaborators

March 12, 2026

Contents

1 Introduction **2**

2 Installation and Environment **2**
2.1 Beginner Setup Checklist 2
2.2 Test Configuration 3

3 Command-Line Overview **4**

4 Configuration and YAML Fields **4**
4.1 Configuration Formats (Python, YAML, or JSON) 4
4.2 Single-Crystal Variations (`nmr-field`, `nmr-vary-*`) 5
4.3 Powder Spectra (Powder CLIs) 5
4.4 Multisite Spectra 5
4.5 Fitting (Frequency-Swept, Multisite or Powder) 6

5 Single-Crystal Simulations **6**

6 Powder and Multisite Simulations **7**

7 Fitting Workflows **7**

8 Testing **7**
8.1 Prerequisites 7
8.2 Running 7
8.3 Conventions and Outputs 8

9 Physics Background **8**
9.1 Zeeman Hamiltonian 8
9.2 Raising and Lowering Operators 8
9.3 Spin Operators (I=3/2 example) 8
9.4 Quadrupolar Hamiltonian 9

9.5 Hyperfine Interaction and Total Hamiltonian 9
9.6 Exact Diagonalization Procedure (Vectorized) 9
9.7 Powder spectra from internal field distributions 10
9.8 From Equations to Code 12
9.9 Second-Order Perturbation Expressions 12
9.10 Isotope Data 12

10 Validation Against Published Data **12**

11 References **13**

1 Introduction

pySimNMR simulates solid-state NMR spectra using exact diagonalization (ED) of the nuclear spin Hamiltonian with optional second-order perturbation expressions when valid. The package supports single-crystal variations (field, angle, and parameter trends), powder patterns, multisite spectra, and basic fitting workflows. The ED core is vectorized to evaluate many Hamiltonians efficiently (e.g., powder orientations, variation grids) without Python loops.

2 Installation and Environment

Use a Python $\geq$ 3.10 virtual environment so the CLIs match the tested toolchain.

```
python3 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install -e .
# Windows PowerShell:
#   py -m venv .venv
#   .\.venv\Scripts\Activate.ps1
# Windows Command Prompt:
#   py -m venv .venv && .\.venv\Scripts\activate.bat
#   python -m pip install --upgrade pip
#   pip install -e .
```

2.1 Beginner Setup Checklist

New users with minimal Python experience can follow this linear workflow:

1. Install Python 3.10+ from python.org, enabling the "Add Python to PATH" option on Windows.

2. Download the pySimNMR source (clone via `git clone <repo>` or extract the ZIP archive).

3. Open a terminal in the project root and create a virtual environment:

```
python3 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install -e .
# Windows PowerShell:
#   py -m venv .venv
#   .\.venv\Scripts\Activate.ps1
#   pip install --upgrade pip
#   pip install -e .
# Windows Command Prompt:
#   py -m venv .venv && .\.venv\Scripts\activate.bat
#   python -m pip install --upgrade pip
#   pip install -e .
```

4. Run a smoke test (no GUI) to confirm the install:

```
nmr-freq-powder \
  --config examples/freq_powder_modern.py \
  --no-show --save-fig output/test_powder.png
```

   The CLI prints progress and writes `output/test_powder.png`. Repeat with any Python, YAML, or JSON config under `examples/` or `examples/reference/` to explore other workflows.

2.2 Test Configuration

Tests can be toggled with a YAML configuration file (default: `tests/test_config.yaml`). Each key mirrors a CLI family; all entries default to `true`:

```
multisite:
  single_isotopes: true
  three_isotopes: true
powder:
  freq: true
  field: true
fit:
  multisite: true
  powder: true
vary:
  freq_vs_field: true
  elevels_vs_field: true
  freq_vs_eta: true
  freq_vs_angle: true
freq:
  multisite_ed: true
freq_fit:
  multisite: true
field_spec:
  multisite: true
field_fit:
```

```
  multisite: true
field_powder:
  modern: true
field_powder_fit:
  modern: true
freq_powder:
  modern: true
freq_powder_fit:
  modern: true
```

Use `--test-config` to point to a custom file:

```
pytest --test-config path/to/my_test_config.yaml -q
```

3 Command-Line Overview

The package exposes dedicated field/frequency/variation commands via console entry points.

- Variations: `nmr-field`, `nmr-elevels`, `nmr-vary-eta`, `nmr-vary-angle`.
- Hyperfine powder builder: `nmr-hyperfine --config examples/hyperfine_distribution_xy_pla`
- Multisite spectra and fits: `nmr-freq`, `nmr-field-spec`, `nmr-freq-fit`, `nmr-field-fit`, plus the powder CLIs (`nmr-freq-powder`, `nmr-field-powder`, and their fitters).
- Headless: add `--no-show` and/or `--save-fig out.png` to any plotting command.

4 Configuration and YAML Fields

This section summarizes the most common YAML fields used by the CLI tools. See commented examples under `examples/` and `examples/reference/`.

4.1 Configuration Formats (Python, YAML, or JSON)

All CLI entry points accept configuration files encoded as Python modules (preferred), YAML, or JSON. Python modules expose a top-level `CONFIG` dictionary and are the recommended format because they support inline comments and expressions. YAML is used in some legacy examples because comments aid readability; JSON works identically. The repository contains a JSON example (`examples/reference/multisite/freq_spectrum.json`) to illustrate this:

```
nmr-freq \
  --config examples/reference/multisite/freq_spectrum.json \
  --no-show --save-fig output/multisite_freq_json.png
```

The JSON file mirrors the YAML variant line-for-line; choose whichever format best suits your tooling.

4.2 Single-Crystal Variations (`nmr-field`, `nmr-vary-*`)

Common fields:

- **isotope**: nucleus label, e.g. `75As`.
- **phi, theta, psi**: ZXZ Euler angles in radians (orientation of EFG and K relative to lab frame).
- **Ka, Kb, Kc**: shift tensor principal values (percent).
- **vQ_MHz**, **eta**: quadrupole principal value (MHz) and asymmetry (unitless).
- Field variations: **B_min_T**, **B_max_T**, **B_points**, **B_axis** (`x`,`y`,`z`).
- η variations: **eta_min**, **eta_max**, **eta_points**; fixed **B0_T**.
- Angle variations: **angle_name** (`phi`, `theta`, `psi`), range/points, and fixed values for the other two angles in degrees.

Older YAML names such as `min_field`/`max_field`, `vc`, or `phi_z_deg` are still accepted; the CLI normalizes them onto the canonical keys above so existing configs do not need to be rewritten.

4.3 Powder Spectra (Powder CLIs)

Common fields (lists are per-site):

- **isotope_list**, **site_multiplicity_list**.
- **Ka_list**, **Kb_list**, **Kc_list**; **vc_list**, **eta_list**.
- Frequency sweep: **H0** (Tesla), **min_freq**, **max_freq**, **n_freq_points**.
- Field sweep: **f0** (MHz), **min_field**, **max_field**, **n_field_points**.
- Broadening: **convolution_function_list** (`gauss` or `lor`), **conv_FWHM_list**, **conv_vQ_FWHM_li**
- ED control: **sim_type** (`exact diag` or `2nd order`), **mtx_elem_min**, **recalc_random_samples**, **n_samples**.

4.4 Multisite Spectra

Fields mirror powder lists but for single-crystal orientations:

- **isotope_list**, **site_multiplicity_list**, **Ka/Kb/Kc_list**, **vc_list**, **eta_list**.
- **H0** (frequency sweep) or **f0** (field sweep).

- Orientation per site: **phi_z_deg_list**, **theta_x_prime_deg_list**, **psi_z_prime_deg_list**.
- Broadening: **convolution_function_list**, **conv_FWHM_list**, **conv_vQ_FWHM_list**.
- ED control: **matrix_element_cutoff** (transition pruning).

4.5 Fitting (Frequency-Swept, Multisite or Powder)

Parameter lists accept multiple forms: `[const]` (fixed), `[val, vary]` (free), `[val, vary, min, max]` (bounded), or `['expr']` (constrained equal). Common fields:

- **H0** (list for fit), **amplitude_list** (site weights).
- **Ka/Kb/Kc**, **vc**, **eta** (lists of lists for fits).
- **FWHM_list**, **FWHM_vQ_list**, **line_shape_func_list**.
- ED control: **mtx_elem_min**.
- Experimental data: **exp_data_file**, delimiter, header lines, scaling/offsets.

5 Single-Crystal Simulations

Configure with Python modules that expose a top-level `CONFIG` dict (or YAML/JSON if preferred; all reside under `examples/`) and run:

```
nmr-field --config examples/freq_vs_field.py --out output/freq_vs_field
```

Other variation commands follow the same pattern (levels vs field, η scans, angle scans). Configuration fields correspond to isotopes, Euler angles, shift tensor components $K_{a,b,c}$, and quadrupole parameters (ν_Q, η). For full frequency-swept spectra at fixed H_0, use:

```
nmr-freq --config examples/reference/multisite/freq_spectrum.py --out
   output/freq_spectrum_multisite
```

When experimental data are available, the companion fitter leverages the same CONFIG style:

```
nmr-freq-fit --config examples/freq_fit_multisite.py --out output/
   freq_fit_multisite --max-nfev 200
```

Field-swept multisite spectra are handled by:

```
nmr-field-spec --config examples/reference/multisite/field_spectrum.py --
   out output/field_spectrum_multisite
```

with the companion fitter:

```
nmr-field-fit --config examples/field_fit_multisite.py --out output/
   field_fit_multisite --max-nfev 200
```

Powder frequency spectra follow the same CONFIG style:

```
nmr-freq-powder --config examples/freq_powder_modern.py --out output/
   freq_powder_modern
```

and the fitting counterpart:

```
nmr-freq-powder-fit --config examples/freq_fit_powder_modern.py --out
   output/freq_powder_fit --max-nfev 200
```

Powder field spectra:

```
nmr-field-powder --config examples/field_powder_modern.py --out output/
   field_powder_modern
nmr-field-powder-fit --config examples/field_fit_powder_modern.py --out
   output/field_powder_fit --max-nfev 200
```

6 Powder and Multisite Simulations

Powder spectra integrate over random orientations; ED mode uses vectorized rotation matrices cached on disk. Multisite tools sum site contributions with independent parameters (K, ν_Q, η, orientations, broadening). See commented examples in `examples/reference/`.

7 Fitting Workflows

The CLI `nmr-freq-fit` wraps lmfit to adjust amplitude/shift/EFG parameters against user-supplied data, and the field/powder fitters reuse the same patterns. Parameter lists accept fixed values, free parameters, bounded parameters, or constraints by name so every workflow behaves consistently.

8 Testing

We use smoke tests to exercise CLI entry points in headless mode and verify basic end-to-end behavior without validating physics-level results.

8.1 Prerequisites

- Install development dependencies: `pytest` and `pyyaml` (tests write and read YAML configs). Easiest: `pip install -e '.[test]'` (quote the extras on zsh).

8.2 Running

From the repository root:

```
pytest -q                  # run all tests
pytest -m "not␣slow" -q    # skip slower powder/fitting tests
pytest -m vary -q          # only run nmr-field/nmr-vary-* CLI tests
```

```
pytest -m freq -q          # run single-crystal frequency simulations
pytest -m freq_fit -q      # run the frequency-spectrum fitting CLI tests
pytest -m field_spec -q    # run the field-spectrum simulation CLI tests
pytest -m field_fit -q     # run the field-spectrum fitting CLI tests
make test                  # equivalent to `pytest -q`
```

8.3 Conventions and Outputs

- Tests save figures under a per-test temporary directory, with descriptive names (e.g., `multisite_freq_75As.png`, `multisite_freq_59Co.png`).
- Tests also direct any text exports (`sim_export_file`) into the same temporary directory (e.g., `multisite_freq_75As.txt`).
- Tests run headlessly: `--no-show` is always used, and `--save-fig` names are explicit.
- Minimal grids are used for speed (e.g., narrow frequency ranges and small histogram bins).

9 Physics Background

9.1 Zeeman Hamiltonian

$$\mathcal{H}_Z = \gamma\,\hbar\,\vec{\mathbf{H}}_{\mathbf{0}} \cdot \hat{\mathbf{I}}\,. \tag{1}$$

For $\vec{\mathbf{H}}_{\mathbf{0}} = H_0\,\hat{z}$: $\mathcal{H}_Z = \gamma\,\hbar\,H_0\,\hat{I}_z$.

9.2 Raising and Lowering Operators

$$I^{\pm} = I_x \pm i I_y, \quad I_x = \tfrac{1}{2}(I^+ + I^-), \quad I_y = \tfrac{1}{2i}(I^+ - I^-)\,. \tag{2}$$

See Slichter [1] for operator identities.

9.3 Spin Operators (I=3/2 example)

For illustration, the spin operators for $I = \frac{3}{2}$ read

$$\hat{I}_z = \begin{pmatrix} \frac{3}{2} & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & -\frac{1}{2} & 0 \\ 0 & 0 & 0 & -\frac{3}{2} \end{pmatrix}, \quad \hat{I}_x = \begin{pmatrix} 0 & \frac{\sqrt{3}}{2} & 0 & 0 \\ \frac{\sqrt{3}}{2} & 0 & 1 & 0 \\ 0 & 1 & 0 & \frac{\sqrt{3}}{2} \\ 0 & 0 & \frac{\sqrt{3}}{2} & 0 \end{pmatrix}, \quad \hat{I}_y = \begin{pmatrix} 0 & -i\frac{\sqrt{3}}{2} & 0 & 0 \\ i\frac{\sqrt{3}}{2} & 0 & -i & 0 \\ 0 & i & 0 & -i\frac{\sqrt{3}}{2} \\ 0 & 0 & i\frac{\sqrt{3}}{2} & 0 \end{pmatrix}. \tag{3}$$

9.4 Quadrupolar Hamiltonian

$$\mathcal{H}_Q = \frac{h\,\nu_z}{6}\left[3\hat{I}_z^2 - I^2 + \eta\,(\hat{I}_x^2 - \hat{I}_y^2)\right], \quad \nu_Q = \nu_z\sqrt{1-\eta^2/3}\,. \tag{4}$$

9.5 Hyperfine Interaction and Total Hamiltonian

$$\mathcal{H}_{\rm hf} = \gamma\,\hbar\,\hat{\mathbf{I}}\cdot\mathbb{A}\cdot\hat{\mathbf{S}}\,, \tag{5}$$

$$\mathcal{H}_{\rm tot} = \mathcal{H}_Z + \mathcal{H}_{\rm hf} + \mathcal{H}_Q\,. \tag{6}$$

9.6 Exact Diagonalization Procedure (Vectorized)

This section outlines the ED workflow used in pySimNMR to compute levels and allowed transitions efficiently across sweeps and orientation grids.

We build the Hamiltonian in a way that allows rotations of the principal axes of the shift (hyperfine) tensor and EFG tensor. In the current implementation these principal axes are locked (coincident), though separating them with a second set of Euler angles for the EFG is straightforward in future revisions. The approach is energy/frequency domain (no time-domain sequence simulation).

The method leverages NumPy vectorization to solve many eigenproblems at once. For example, we can feed arrays of field values H_0, Euler angles, or η and compute eigenpairs in batches without Python loops, dramatically reducing compute time for powder patterns or field sweeps.

Hyperfine terms are separated into a field-dependent shift tensor $\mathbb{K}$ and field-independent contributions (internal field Zeeman interaction with $\vec{H}_{\rm int}$ and the quadrupolar Hamiltonian $\mathcal{H}_Q$). We assume the external field does not modify $\vec{H}_{\rm int}$.

We construct the spin operators I_z, I_x, I_y, and $I_\pm$ and rotation operators in 3D and spin space. The 3D rotation matrices for rotations about x, y, z are the familiar forms, for example

$$r_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}, \quad r_y = \begin{pmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{pmatrix}, \quad r_z = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{7}$$

In practice we use arrays of such matrices with shape $(N, 3, 3)$ for vectorized rotation of many orientations.

Spin-space rotation matrices are defined by

$$\mathcal{R}_{x,y,z} = \exp\!\big(i\,\theta\,\hat{I}_{x,y,z}\big)\,. \tag{8}$$

For performance, we compute $\mathcal{R}$ via eigen-decomposition rather than `expm`: if $\hat{I}_\alpha$ has eigen-decomposition $UDU^\dagger$, then $\exp(i\,\theta\,\hat{I}_\alpha) = U\,\exp(i\,\theta D)\,U^\dagger$. In code:

```
e, U = np.linalg.eig(1j*theta[:,None,None]*Ix)
Ui = np.transpose(np.conj(U), (0,2,1))
expD = np.eye(dim) * np.exp(e)[:,None,:]
Rx = U @ expD @ Ui  # shape: (N, dim, dim)
```

The shift tensor

$$\mathbb{K}^{\text{init}} = \text{diag}(K_a, K_b, K_c)/100 \tag{9}$$

is rotated by Euler angles ϕ_z, $\theta_{x'}$, $\psi_{z'}$ using real-space rotations r and $r_i = r^\top$:

$$\mathbb{K} = r\,\mathbb{K}^{\text{init}}\,r_i\,. \tag{10}$$

The Zeeman term then reads

$$\mathcal{H}_Z = \gamma\,(\mathbb{1} + \mathbb{K}) \cdot \vec{H}_0\,, \quad \vec{H}_0 = H_0\,\hat{z}\,. \tag{11}$$

An internal Zeeman term is included by rotating the internal field vector in the same way,

$$\vec{H}_{\text{int}} = r\,\vec{H}^{\text{init}}_{\text{int}}\,, \qquad \mathcal{H}_{Z,\text{int}} = \gamma\,\vec{H}_{\text{int}} \cdot \hat{\vec{I}}\,. \tag{12}$$

The quadrupolar Hamiltonian is rotated in spin space using the spin-space rotations $\mathcal{R}, \mathcal{R}_i$:

$$\mathcal{H}_Q = \mathcal{R}\,\mathcal{H}^{\text{init}}_Q\,\mathcal{R}_i\,, \qquad \mathcal{R} = \mathcal{R}(\psi_{z'})\,\mathcal{R}(\theta_{x'})\,\mathcal{R}(\phi_z)\,. \tag{13}$$

The total Hamiltonian is

$$\mathcal{H}_{\text{tot}} = \mathcal{H}_Z + \mathcal{H}_{Z,\text{int}} + \mathcal{H}_Q\,. \tag{14}$$

We then diagonalize batched arrays of $\mathcal{H}_{\text{tot}}$ via `numpy.linalg.eig`, obtaining eigenvalues (levels) and column-wise eigenvectors. Allowed transitions and their intensities follow from matrix elements of I_x or equivalently $I_\pm$:

$$P = \left(U^\dagger\,\hat{I}^+\,U\right)^2\,. \tag{15}$$

By indexing the non-negligible elements of P, we associate each transition with its initial/-final levels and compute resonant frequencies (energy differences) and relative intensities.

9.7 Powder spectra from internal field distributions

A powder spectrum can be assembled from a probability distribution over internal fields. Given PAS-aligned samples $\vec{H}^{(i)}_{\text{int}}$ with weights w_i, the helper `freq_spectrum_from_internal_field_distr` loops over the samples, invokes `freq_spec_ed` for each, and accumulates the weighted spectra. Euler-angle stacks `euler_angles_rad` can be provided when the PAS orientation varies across samples; the matrices returned by `generate_r_matrices` and `generate_r_spin_matrices` are reused under the hood for each internal field draw. Weights are normalized automatically.

```
from pysimnmr.core import SimNMR
from pysimnmr.hyperfine_distribution import
   freq_spectrum_from_internal_field_distribution

sim = SimNMR('1H')
freq_axis = np.linspace(18.0, 24.0, 512)
hint_samples = np.array([[0.0, 0.0, 0.0], [0.0, 0.0, 0.05]])
weights = np.array([0.7, 0.3])

spectrum = freq_spectrum_from_internal_field_distribution(
    sim,
    freq_axis,
    hint_pas_samples_T=hint_samples,
    weights=weights,
    Ka=0.0, Kb=0.0, Kc=0.0,
    vQ_MHz=0.1, eta=0.0, H0_T=0.5,
    FWHM_MHz=0.05,
)
```

The sampled internal fields can be visualized interactively using `save_internal_field_distribution_htm` which emits a Plotly 3D scatter plot colored by the supplied weights. Example scripts (`examples/internal_field_powder_xy_plane.py`) demonstrate a 75As distribution restricted to the xy-plane at zero applied field. The same parameters are encoded in the YAML file `examples/hyperfine_distribution_xy_plane.py`, which can be executed via `nmr-hyperfine --config` A typical configuration specifies probability distributions explicitly, e.g.

```
hyperfine_distribution:
  samples: 1000
  seed: 20251007
  plane: xy
  angle_distribution:
    type: uniform
  magnitude_distribution:
    type: gaussian
    mean_T: 0.20
    sigma_T: 0.04
  weights:
    type: equal

# Uniform directions with fixed magnitude in the xy plane:
# hyperfine_distribution:
#   samples: 1000
#   seed: 20251007
#   plane: xy
#   angle_distribution:
#     type: uniform
#   magnitude_distribution:
#     type: delta
#     value_T: 0.20
#   weights:
#     type: equal
```

Explicit vectors with their own probabilities can be supplied instead via the `vectors` and `weights` keys. Custom vector generators can also be registered by setting `allow_custom_generators: tru` in the YAML and pointing `custom_generator.module/function` to user code that returns vectors (and optional weights).

9.8 From Equations to Code

The following objects in the implementation correspond to the equations above (see `pysimnmr/core.py`):

- Real-space rotations: `generate_r_matrices(phi, theta, psi)` produce $r, r^\top$.
- Spin-space rotations: `generate_r_spin_matrices(phi, theta, psi)` produce $\mathcal{R}, \mathcal{R}^\dagger$.
- Level sweeps: `elevels_vs_field_ed(...)` assembles $\mathcal{H}_{\text{tot}}$ across H_0 and diagonalizes.
- Transition frequencies: `exact_diag(...)` returns the allowed transitions/probabilities directly from $U^\dagger I_x U$ (the deprecated `freq_prob_trans_ed` helper remains for historical reference only).
- Spectrum constructors: `freq_spec_ed_mix(...)` (frequency sweeps) and `field_spec_edpp(...)` (field sweeps) wrap `exact_diag` output with histogramming/broadening.
- CLI: `nmr-field`, `nmr-elevels`, `nmr-vary-eta`, `nmr-vary-angle`, `nmr-freq`, `nmr-field-spec`, `nmr-freq-powder`, and the matching fitters all share the same YAML/Python config model.

9.9 Second-Order Perturbation Expressions

For regimes where $\nu_L \gg \nu_Q$, second-order expressions are useful and fast. We retain classical formulas for powder patterns and central-transition shifts following Baugher et al. [2] and Taylor et al. [3]. Use ED near level crossings or when $\eta \neq 0$ effects are significant.

9.10 Isotope Data

Gyromagnetic ratios, quadrupole moments, and spin values follow Harris et al. [4]. See `pysimnmr/isotopeDict.py`.

10 Validation Against Published Data

The table below summarizes which CLIs and example configs have been exercised against experimental data from the literature. **Status codes:** Partial – simulation ran and reproduced the general lineshape but some features did not agree; retained as a starting point for refinement. Pending – config exists with literature parameters but has not yet been compared against a digitized experimental spectrum.

11 References

References

[1] C. P. Slichter, *Principles of Magnetic Resonance*, 3rd ed. (Springer, 1996).

[2] J. F. Baugher, P. C. Taylor, T. Oja, and P. J. Bray, The Journal of Chemical Physics **50**, 4914 (1969).

[3] P. C. Taylor, J. F. Baugher, and H. M. Kriz, Chemical Reviews **75**, 203 (1975).

[4] R. K. Harris *et al.*, Pure and Applied Chemistry **73**, 1795 (2001).

[5] A. M. Mounce *et al.*, Physical Review B **93**, 024419 (2016).

Compound	Nucleus	CLI	Config file	Status
PuMeCl	^{35}Cl, ^{37}Cl	`nmr-field-powder`	`examples/reference/powder/field_spectrum_Mounce_2016.py`	Partial – general lineshape reproduced; small features differ, likely due to `mtx_elem_min` being too low. See Ref. [5]
PuB_4	^{11}B	`nmr-field-powder`	`examples/reference/powder/field_spectrum_borides.py`	Pending – placeholder parameters; validate against Blackwell et al.
$BaFe_2As_2$	^{75}As	`nmr-freq`	`examples/reference/multisite/freq_spectrum.py`	Pending – parameters consistent with iron-pnictide literature; validate against Kitagawa et al.
75As / 73Ge (synthetic)	^{75}As, ^{73}Ge	`nmr-hyperfine`	`examples/hyperfine_distribution_xy_plane.py`, `hyperfine_uniform_xy_plane_73Ge.py`	Pending – synthetic demo of internal-field workflow; needs a published reference compound for validation

Table 1: Summary of reference configs and their validation status against published experimental NMR data. See `TODO.md` for the planned validation roadmap.